

Notification documentation

Step 1 — Map the code that mentions notifications

Files Referencing "notification" or "notifications" in lib

PRESENTATION LAYER

Controllers:

- notifications_controller.dart
- notifications_controller_v2.dart

Providers:

- notifications_providers.dart

Screens:

- notifications_screen.dart 🌟 (currently in use for social)
- notification_settings_screen.dart
- social_onboarding_notifications_screen.dart
- notifications_screen_supabase.dart
- notifications_screen_v2.dart
- notification_settings_screen.dart

Widgets:

- achievement_notification_widget.dart
 - achievement_notification.dart
 - rewards_notification_banner.dart
 - notification_helper.dart
-

DOMAIN LAYER

Entities:

- notification.dart

Repositories (Interfaces):

- notifications_repository.dart
-

DATA LAYER

Models:

- notification_model.dart
- reward_notification_model.dart
- notification_model.dart
- achievement_notifications.dart
- game_notifications.dart
- social_notifications.dart
- notifications.dart
- notifications_unified.dart

Datasources:

- notifications_datasource.dart

Repositories (Implementations):

- notifications_repository_impl.dart
 - notifications_repository.dart
 - achievement_notifications_repo.dart
 - game_notifications_repo.dart
 - social_notifications_repo.dart
 - notifications_repo.dart
 - notifications_unified_repo.dart
-

SERVICES LAYER

- notification_service.dart
 - push_notification_service.dart
 - game_notifications_service.dart
 - achievement_notification_service.dart
 - social_notifications_service.dart
 - social_notifications_service.dart.disabled
-

UTILITIES/SHARED

- analytics_constants.dart (notification constants)
 - analytics_service.dart (notification tracking)
 - analytics_widgets.dart (ScrollNotification)
 - notification_type_enums.dart
 - notification_helper.dart.disabled
 - notification_helpers.dart
-

Summary: There is a full `notifications` feature module at notifications with Clean Architecture structure. The friend requests notification is currently displayed through `notifications_screen.dart`, which shows hardcoded data but has a badge that queries the `friendships` table directly.

Step 2 — Explain the end-to-end flow (UI → Domain → Data)

NOTIFICATIONS FLOW ANALYSIS

1. IMPORTS CHAIN

```

// UI Layer
lib/screens/notifications/notifications_screen_v2.dart
├─ import 'package:flutter_riverpod/flutter_riverpod.dart';
├─ import '../features/notifications/data/notifications_repository.dart';
└─ import '../features/notifications/presentation/providers/notifications_providers.dart';

// Provider Layer
lib/features/notifications/presentation/providers/notifications_providers.dart
├─ import 'package:flutter_riverpod/flutter_riverpod.dart';
├─ import 'package:supabase_flutter/supabase_flutter.dart';
├─ import '../data/notifications_repository.dart';
└─ import '../controllers/notifications_controller_v2.dart';

// Controller Layer
lib/features/notifications/presentation/controllers/notifications_controller_v2.dart
├─ import 'package:flutter_riverpod/flutter_riverpod.dart';
└─ import '../data/notifications_repository.dart';

// Repository/Datasource Layer (COMBINED)
lib/features/notifications/data/notifications_repository.dart
└─ import 'package:supabase_flutter/supabase_flutter.dart';

```

2. METHODS CHAIN

UI → Provider

```

// File: lib/screens/notifications/notifications_screen_v2.dart:49
final state = ref.watch(notificationsControllerProvider(userId));
// Returns: NotificationsState

```

Provider → Controller Constructor

```

// File: lib/features/notifications/presentation/providers/notifications_providers.dart:17

```

```
return NotificationsController(  
  repository: repository,  
  userId: userId,  
);  
// Returns: NotificationsController instance
```

Controller → Repository (Initial Load)

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:103  
final notifications = await _repository.list(  
  userId: _userId,  
  limit: 20,  
  cursor: refresh ? null : state.cursor,  
);  
// Returns: Future<List<NotificationItem>>
```

Controller → Repository (Mark as Read)

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:143  
await _repository.markRead(notificationId: notificationId);  
// Returns: Future<void>
```

Controller → Repository (Realtime)

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:62  
_realtimeSub = _repository.subscribeUserNotifications(_userId).listen(...)  
// Returns: Stream<NotificationItem>
```

3. DATASOURCE QUERY SNIPPETS

List Notifications (Keyset Pagination)

```
// File: lib/features/notifications/data/notifications_repository.dart:264-296
var query = _client
  .from('notifications')
  .select()
  .eq('user_id', userId);

// Apply type filter
if (typeFilter != null) {
  query = query.eq('type', typeFilter);
}

// Apply keyset pagination cursor
if (cursor != null) {
  final cursorTime = cursor.createdAt.toIso8601String();
  final cursorId = cursor.id;

  query = query.or(
    'created_at.lt.$cursorTime,and(created_at.eq.$cursorTime,id.lt.$cursorId)'
  );
}

// Order and limit
final response = await query
  .order('created_at', ascending: false)
  .order('id', ascending: false)
  .limit(limit);
```

Mark as Read

```
// File: lib/features/notifications/data/notifications_repository.dart:376-388
await _client
  .from('notifications')
  .update({
    'is_read': true,
    'read_at': DateTime.now().toIso8601String(),
    'updated_at': DateTime.now().toIso8601String(),
```

```
  })  
  .eq('id', notificationId);
```

Mark All as Read

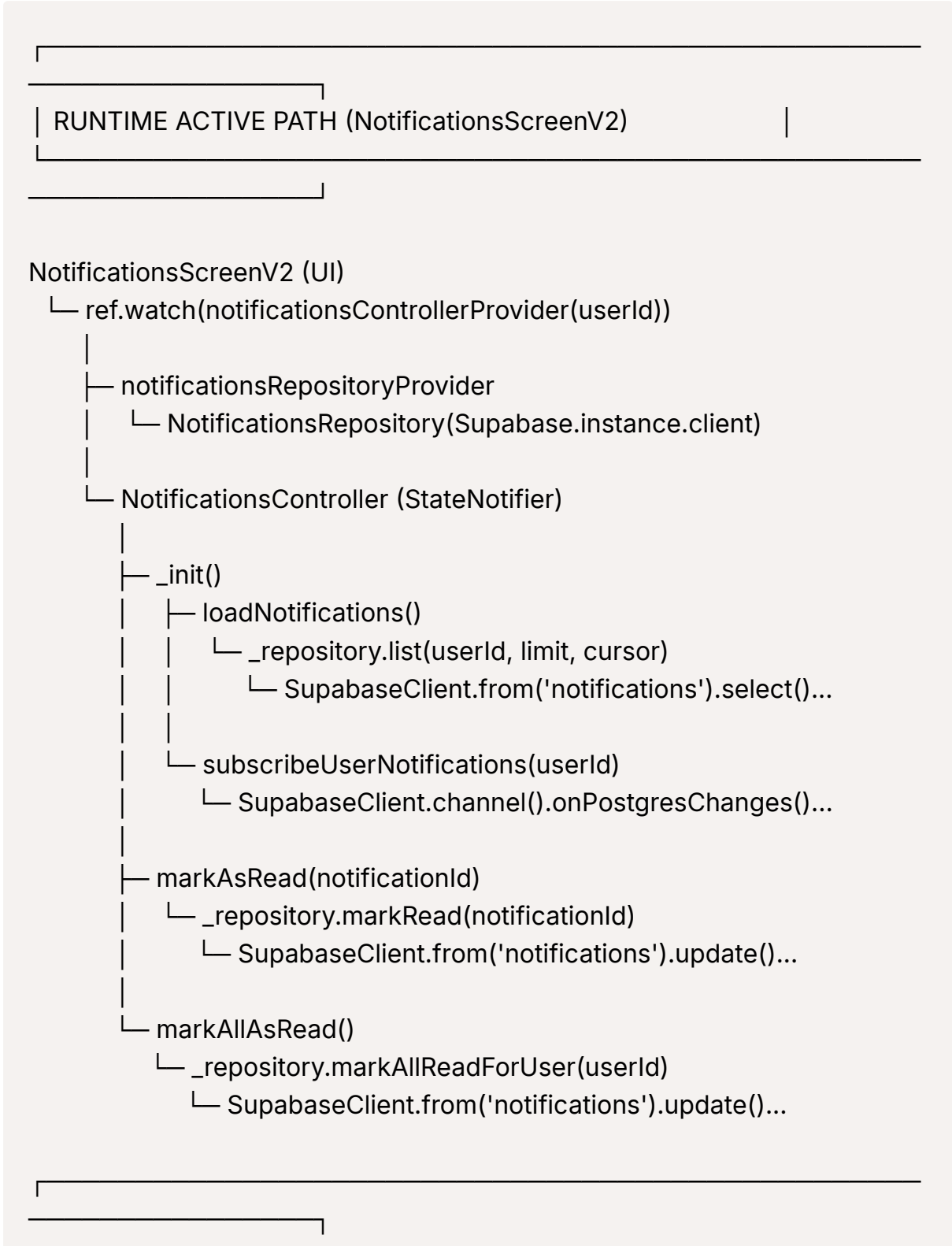
```
// File: lib/features/notifications/data/notifications_repository.dart:421-434  
await _client  
  .from('notifications')  
  .update({  
    'is_read': true,  
    'read_at': now,  
    'updated_at': now,  
  })  
  .eq('user_id', userId)  
  .eq('is_read', false); // Only update unread
```

Realtime Subscription

```
// File: lib/features/notifications/data/notifications_repository.dart:481-524  
final channel = _client  
  .channel(channelKey)  
  .onPostgresChanges(  
    event: PostgresChangeEvent.insert,  
    schema: 'public',  
    table: 'notifications',  
    filter: PostgresChangeFilter(  
      type: PostgresChangeFilterType.eq,  
      column: 'user_id',  
      value: userId,  
    ),  
    callback: (payload) {  
      final notification = NotificationItem.fromMap(payload.newRecord);  
      controller.add(notification);  
    },  
  )  
  .onPostgresChanges(  
    event: PostgresChangeEvent.update,
```

```
...
)
.subscribe();
```

4. ASCII CALL DIAGRAM



DATA FLOW

Supabase DB (notifications table)

↓ SELECT (list)

NotificationItem.fromMap(json)

↓

List<NotificationItem>

↓

NotificationsState.copyWith(notifications: [...])

↓

UI renders ListView with notification cards

Supabase Realtime (INSERT/UPDATE events)

↓

NotificationItem.fromMap(payload.newRecord)

↓

Stream<NotificationItem>

↓

Controller updates state.notifications

↓

UI auto-updates via ref.watch()

5. UNUSED/LEGACY FILES DETECTED

NOT Referenced by Active Path:

✗ lib/features/social/presentation/screens/placeholders/notifications_screen.dart

(Shows hardcoded data, uses friendships table badge only)

✗ lib/screens/notifications/notifications_screen_supabase.dart

(Legacy implementation)

✗ lib/screens/notifications/notification_settings_screen.dart

(Old settings screen)

✗ lib/features/notifications/presentation/controllers/notifications_controller.dart

(Old controller, replaced by _v2)

✗ lib/features/notifications/domain/entities/notification.dart

(Not used - repository uses NotificationItem directly)

✗ lib/features/notifications/domain/repositories/notifications_repository.dart

(Interface not used - repository is concrete implementation)

✗ lib/features/notifications/data/repositories/notifications_repository_impl.dart

(Not used - NotificationsRepository is the impl)

✗ lib/features/notifications/data/datasources/notifications_datasource.dart

(Not used - repository calls Supabase directly)

✗ lib/features/notifications/data/models/notification_model.dart

(Not used - repository uses NotificationItem)

✗ lib/repositories/notifications_unified_repo.dart

(Legacy unified repo)

✗ lib/repositories/notifications_repo.dart

(Legacy repo)

✗ lib/repositories/achievement_notifications_repo.dart

(Separate concern, not in main notifications flow)

✗ lib/repositories/game_notifications_repo.dart

(Separate concern, not in main notifications flow)

✗ lib/repositories/social_notifications_repo.dart

(Separate concern, not in main notifications flow)

✗ lib/models/notifications.dart

(Legacy model)

✗ lib/models/notifications_unified.dart
(Legacy model)

✗ lib/models/achievement_notifications.dart
(Legacy model)

✗ lib/models/game_notifications.dart
(Legacy model)

✗ lib/models/social_notifications.dart
(Legacy model)

Architecture Note: This app **collapsed Clean Architecture layers** - the repository file combines:

- Domain entities (`NotificationItem`)
- Repository interface (methods)
- Repository implementation (Supabase calls)
- Datasource (direct `_client.from()` queries)

6. UNREAD COUNT SOURCE

Computed in Controller:

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:114-117
final unreadCount = updatedList.where((n) => !n.isRead).length;

state = state.copyWith(
  notifications: updatedList,
  unreadCount: unreadCount,
);
```

Method: In-memory count of `!isRead` items from loaded notifications list.

Alternative (Not Used): Could query `COUNT(*)` from database:

```
SELECT COUNT(*) FROM notifications
WHERE user_id = ? AND is_read = false;
```

Friend Requests Badge (Separate System):

```
// File: lib/features/social/presentation/screens/friend_requests_screen.dar
t:24-33
final pendingFriendRequestsCountProvider = FutureProvider.autoDispose<int>((ref) async {
  final currentUserId = client.auth.currentUser?.id;
  if (currentUserId == null) return 0;

  final repo = FriendshipsRepo(client);
  return repo.getPendingFriendRequestsCount(currentUserId);
  // Queries: SELECT * FROM friendships WHERE friend_id = ? AND status =
  'pending'
  // Returns: (res as List).length
});
```

SUMMARY

Active Path: NotificationsScreenV2 → notificationsControllerProvider → NotificationsController → NotificationsRepository → SupabaseClient.from('notifications')

Key Observations:

1. **No Clean Architecture separation** - repository is monolithic with all layers
2. **No domain layer interfaces** - direct concrete implementation
3. **Realtime subscription** using Supabase Postgres Changes
4. **Keyset pagination** with (created_at, id) cursor
5. **Unread count** computed in-memory, not via SQL COUNT
6. **Friend requests badge** is separate system querying friendships table

Step 3 — “Open bell” path (badge, initial load, pagination)

A) UNREAD BADGE VALUE SOURCE

Computation & Update Locations

Location 1: Realtime INSERT/UPDATE Handler

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:68-71
final updatedList = [notification, ...state.notifications];
final unreadCount = updatedList.where((n) => !n.isRead).length;

state = state.copyWith(
  notifications: updatedList,
  unreadCount: unreadCount,
);
```

Location 2: Realtime UPDATE Handler

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:80-83
final updatedList = [...state.notifications];
updatedList[index] = notification;
final unreadCount = updatedList.where((n) => !n.isRead).length;

state = state.copyWith(
  notifications: updatedList,
  unreadCount: unreadCount,
);
```

Location 3: Initial Load & Pagination

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:114-117
```

```

final updatedList = refresh
  ? notifications
  : [...state.notifications, ...notifications];
final unreadCount = updatedList.where((n) ⇒ !n.isRead).length;

state = state.copyWith(
  notifications: updatedList,
  cursor: notifications.last.cursor,
  isLoading: false,
  hasMore: notifications.length >= 20,
  unreadCount: unreadCount,
  error: null,
);

```

Location 4: Delete Notification

```

// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:171-174
final updatedList = state.notifications
  .where((n) ⇒ n.id != notificationId)
  .toList();
final unreadCount = updatedList.where((n) ⇒ !n.isRead).length;

state = state.copyWith(
  notifications: updatedList,
  unreadCount: unreadCount,
);

```

Formula: `unreadCount = updatedList.where((n) ⇒ !n.isRead).length`

- Iterates entire loaded notifications list
- Counts items where `isRead == false`
- In-memory computation (not a database query)

B) INITIAL LOAD FLOW

Entry Point: Constructor → `_init()`

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:50-56
NotificationsController({
  required NotificationsRepository repository,
  required String userId,
}) : _repository = repository,
    _userId = userId,
    super(const NotificationsState()) {
  _init(); // ← Initial load triggered here
}
```

`_init()` Method - Loads First Page

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:58-93
void _init() {
  // Load initial notifications
  loadNotifications(); // ← Calls with default parameters (refresh=false)

  // Subscribe to realtime updates
  _realtimeSub = _repository.subscribeUserNotifications(_userId).listen(
    (notification) {
      // Realtime handlers...
    },
  );
}
```

`loadNotifications()` Method Body

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:95-133
/// Load notifications (initial or refresh)
Future<void> loadNotifications({bool refresh = false}) async {
  if (state.isLoading) return; // ← Guard against concurrent loads

  if (refresh) {
    state = const NotificationsState(isLoading: true); // ← Reset state
  }
}
```

```

    } else {
        state = state.copyWith(isLoading: true, error: null); // ← Keep existing data
    }

    try {
        final notifications = await _repository.list(
            userId: _userId,
            limit: 20,
            cursor: refresh ? null : state.cursor, // ← null on first load, then uses saved cursor
        );

        if (notifications.isEmpty) {
            state = state.copyWith(
                isLoading: false,
                hasMore: false,
            );
        } else {
            final updatedList = refresh
                ? notifications // ← Replace all (refresh)
                : [...state.notifications, ...notifications]; // ← Append (pagination)
            final unreadCount = updatedList.where((n) => !n.isRead).length;

            state = state.copyWith(
                notifications: updatedList,
                cursor: notifications.last.cursor, // ← Cursor set HERE from last item
                isLoading: false,
                hasMore: notifications.length >= 20, // ← If got full page, assume more exist
                unreadCount: unreadCount,
                error: null,
            );
        }
    } catch (e) {
        state = state.copyWith(
            isLoading: false,
            error: e.toString(),

```

```
);  
}  
}
```

Cursor Setting:

```
// Line 119: Cursor is extracted from the LAST notification in the response  
cursor: notifications.last.cursor,  
  
// Cursor is a property on NotificationItem:  
// File: lib/features/notifications/data/notifications_repository.dart:146-149  
NotificationCursor get cursor ⇒ NotificationCursor(  
  createdAt: createdAt,  
  id: id,  
);
```

C) PAGINATION

UI Scroll Listener - Triggers Load More

```
// File: lib/screens/notifications/notifications_screen_v2.dart:34-42  
void _onScroll() {  
  if (_scrollController.position.pixels >=  
    _scrollController.position.maxScrollExtent - 200) { // ← 200px before bottom  
    final userId = _authService.getCurrentUserId();  
    if (userId != null) {  
      ref.read(notificationsControllerProvider(userId).notifier).loadMore(); //  
      ← Calls loadMore()  
    }  
  }  
}
```

Attached in initState:

```
// File: lib/screens/notifications/notifications_screen_v2.dart:24-27  
@override
```

```
void initState() {
  super.initState();
  _scrollController.addListener(_onScroll); // ← Listener attached
}
```

loadMore() Method - Delegates to loadNotifications()

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:135-138
/// Load more notifications (pagination)
Future<void> loadMore() async {
  if (state.isLoading || !state.hasMore) return; // ← Guard against concurrent/unnecessary loads
  await loadNotifications(); // ← Calls with refresh=false (default)
}
```

Repository Call with Cursor

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:107-109
final notifications = await _repository.list(
  userId: _userId,
  limit: 20,
  cursor: refresh ? null : state.cursor, // ← Uses saved cursor from previous page
);
```

Cursor Computation & Storage

```
// File: lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:119
cursor: notifications.last.cursor, // ← Extracts from last item in response
```

Cursor Definition:

```
// NotificationItem has a getter that creates cursor from its own fields:
// File: lib/features/notifications/data/notifications_repository.dart:146-149
```

```
NotificationCursor get cursor ⇒ NotificationCursor(  
  createdAt: createdAt, // ← Timestamp of this notification  
  id: id,               // ← UUID of this notification  
);
```

Cursor Usage in Repository Query:

```
// File: lib/features/notifications/data/notifications_repository.dart:281-288  
if (cursor != null) {  
  // Keyset pagination: created_at < cursor OR (created_at = cursor AND id  
< cursor.id)  
  final cursorTime = cursor.createdAt.toIso8601String();  
  final cursorId = cursor.id;  
  
  query = query.or(  
    'created_at.lt.$cursorTime,and(created_at.eq.$cursorTime,id.lt.$cursorI  
d)'  
  );  
}
```

POTENTIAL BUGS & ISSUES

Bug 1: Cursor Not Advanced on Empty Page

```
// Line 111-116: If notifications.isEmpty, cursor is NOT updated  
if (notifications.isEmpty) {  
  state = state.copyWith(  
    isLoading: false,  
    hasMore: false, // ← Sets hasMore=false, preventing future loads  
  );  
  // ⚠️ But cursor stays at previous value!  
}
```

Impact: Low - `hasMore=false` prevents further pagination attempts

Bug 2: Race Condition - No Request Deduplication

```
// Line 97: Guard only checks isLoading flag
if (state.isLoading) return;

// ⚠️ PROBLEM: If user scrolls rapidly, multiple scroll events can fire
// BEFORE isLoading is set to true. This can trigger multiple concurrent requests.
```

Scenario:

1. Scroll event 1: `isLoading=false` → starts request → async gap before `state = state.copyWith(isLoading: true)`
2. Scroll event 2 (during gap): `isLoading=false` → starts DUPLICATE request
3. Both requests complete, potentially loading same data twice

Fix: Use request ID/timestamp or atomic flag

Bug 3: unreadCount Only Reflects Loaded Items

```
// Line 116: unreadCount computed from in-memory list
final unreadCount = updatedList.where((n) => !n.isRead).length;
```

Problem: If user has 100 unread notifications but only loaded 20, badge shows "20" instead of "100"

Fix: Query database for total unread count:

```
SELECT COUNT(*) FROM notifications
WHERE user_id = ? AND is_read = false;
```

Bug 4: Realtime Updates Don't Recalculate Pagination State

```
// Line 68-72: Realtime INSERT prepends notification
final updatedList = [notification, ...state.notifications];
// ⚠️ But doesn't update hasMore or cursor
```

Problem: If user loads page 1 (20 items), then receives 5 realtime notifications:

- `updatedList.length = 25`

- But `hasMore` still based on original page size
- Cursor might be stale

Impact: Medium - Could cause duplicate items if user paginates after realtime updates



Bug 5: No Optimistic Update for markAsRead

```
// Line 151-156: Waits for realtime subscription to update UI
await _repository.markRead(notificationId: notificationId);
// Update will come via realtime subscription
```

Problem:

- Network latency before realtime event arrives
- UI shows "unread" state for 100-500ms after tap
- Poor UX

Fix: Optimistic update:

```
// 1. Update local state immediately
final index = state.notifications.indexWhere((n) => n.id == notificationId);
if (index != -1) {
  final updated = [...state.notifications];
  updated[index] = updated[index].copyWith(isRead: true);
  state = state.copyWith(notifications: updated);
}
// 2. Then call API
await _repository.markRead(notificationId: notificationId);
```



Good Practices Found

1. **Keyset pagination** - Efficient, handles real-time inserts better than offset
2. **Guard against concurrent loads** - `if (state.isLoading) return;`
3. **Append vs Replace logic** - `refresh ? notifications : [...state.notifications, ...notifications]`
4. **Realtime subscription** - Auto-updates on INSERT/UPDATE
5. **Stateless widget pattern** - Controller is separate from UI

Summary Table

Component	Location	Trigger	Notes
Initial Load	<code>_init()</code> → <code>loadNotifications()</code>	Constructor	Cursor=null, loads first 20
Pagination	<code>_onScroll()</code> → <code>loadMore()</code> → <code>loadNotifications()</code>	Scroll within 200px of bottom	Uses saved cursor
Cursor Set	<code>notifications.last.cursor</code>	After successful load	Extracted from last item
Unread Count	<code>updatedList.where((n) ⇒ !n.isRead).length</code>	After every state change	In-memory count
Race Guard	<code>if (state.isLoading) return;</code>	Before load	⚠ Weak - async gap vulnerability

Step 4 — “Tap item” path (mark read + navigate)

NOTIFICATION ITEM onTap FLOW ANALYSIS

1) onTap Handler Code

Location: `lib/screens/notifications/notifications_screen_v2.dart:304`

```
onTap: () ⇒ _handleNotificationTap(userId, notification),
```

Context: Inside ListTile widget in `_buildNotificationCard()` method

Full Code Block (lines 266-307):

```
child: ListTile(  
  contentPadding: const EdgeInsets.all(16),  
  leading: _getNotificationIcon(notification.type, notification.priority),  
  title: Text(  
    notification.title,  ),  
),
```

```

notification.title,
style: context.textTheme.titleMedium?.copyWith(
  fontWeight: notification.isRead ? FontWeight.normal : FontWeight.bold,
  color: context.colors.onSurface,
),
),
subtitle: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    const SizedBox(height: 4),
    Text(
      notification.message,
      style: context.textTheme.bodyMedium?.copyWith(
        color: context.colors.onSurfaceVariant,
      ),
    ),
    const SizedBox(height: 8),
    Text(
      _formatTime(notification.createdAt),
      style: context.textTheme.bodySmall?.copyWith(
        color: context.colors.onSurfaceVariant,
      ),
    ),
  ],
),
trailing: notification.isRead
  ? null
  : Container(
    width: 8,
    height: 8,
    decoration: BoxDecoration(
      color: context.colors.primary,
      shape: BoxShape.circle,
    ),
  ),
onTap: () ⇒ _handleNotificationTap(userId, notification), // ← HERE
),

```

2) Mark-as-Read Call

Handler Method: `lib/screens/notifications/notifications_screen_v2.dart:391-404`

```
Future<void> _handleNotificationTap(String userId, NotificationItem notification) async {
  // Mark as read
  if (!notification.isRead) {
    await ref
      .read(notificationsControllerProvider(userId).notifier)
      .markAsRead(notification.id);
  }

  // Navigate if action route exists
  if (notification.actionRoute != null && mounted) {
    context.push(notification.actionRoute!);
  }
}
```

Answer: **AWAITS** the Future (line 394: `await ref...markAsRead(...)`)

Controller Method: `lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:155-161`

```
/// Mark notification as read
Future<void> markAsRead(String notificationId) async {
  try {
    await _repository.markRead(notificationId: notificationId);
    // Update will come via realtime subscription
  } catch (e) {
    state = state.copyWith(error: e.toString());
  }
}
```

Flow:

1. UI calls `markAsRead()` with `await`
2. Controller awaits repository call
3. **No optimistic update** - waits for realtime subscription

4. **Error captured in controller** - sets `state.error`
 5. **BUT:** Error is NOT propagated to UI (no rethrow)
-

3) Navigation Call & Route Decision

Location: `lib/screens/notifications/notifications_screen_v2.dart:401-403`

```
// Navigate if action route exists
if (notification.actionRoute != null && mounted) {
  context.push(notification.actionRoute!);
}
```

Route Source: `notification.actionRoute` (NOT `notification.data.route`)

Model Definition: `lib/features/notifications/data/notifications_repository.dart:149`

```
final String? actionRoute;
```

Database Column: `action_route` (mapped from DB in line 192)

```
actionRoute: map['action_route'] as String?,
```

Guards:

1. **Null check:** `notification.actionRoute != null`
2. **Widget mounted check:** `mounted` (prevents navigation after widget disposal)
3. **Non-null assertion:** `notification.actionRoute!` (safe due to null check)

Navigation Method: Uses GoRouter's `context.push()` (not Navigator)

Does NOT use `item.data.route` :

- `item.data` is a `Map<String, dynamic>?` field for arbitrary JSON data
 - Separate from `actionRoute` which is a dedicated nullable String field
-

4) Error Handling

A) Mark-as-Read Errors

Caught in Controller:

`lib/features/notifications/presentation/controllers/notifications_controller_v2.dart:159-160`

```
} catch (e) {  
  state = state.copyWith(error: e.toString());  
}
```

NOT Propagated to UI:

- Exception is caught and stored in `state.error`
- No `rethrow` statement
- UI's `await` completes successfully even if marking as read fails

UI Does NOT Display Error:

- `_handleNotificationTap()` has no try/catch
- No check for `state.error` after await
- User gets NO feedback if mark-as-read fails

B) Navigation Errors

NOT Caught:

```
context.push(notification.actionRoute!); // ← No try/catch
```

Potential Failures:

- Invalid route (404)
- GoRouter exception
- Route requires auth but user is logged out

Would Crash or Show GoRouter Error Page

C) Example of Error Handling in Another Method

For comparison, `_markAllAsRead()` has proper error handling:

`lib/screens/notifications/notifications_screen_v2.dart:406-431`

```
Future<void> _markAllAsRead(String userId) async {  
  try {  
    await ref  
      .read(notificationsControllerProvider(userId).notifier)  
      .markAllAsRead();  
  }
```

```

if (mounted) {
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('All notifications marked as read'),
      behavior: SnackBarBehavior.floating,
    ),
  );
}
} catch (e) { // ← Has try/catch
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Failed to mark all as read: $e'),
        backgroundColor: Colors.red,
        behavior: SnackBarBehavior.floating,
      ),
    );
  }
}
}

```

Why different? `_markAllAsRead()` is a bulk operation where failure is more noticeable and should be communicated.

SUMMARY TABLE

Aspect	Implementation	File:Line
onTap Trigger	<code>onTap: () ⇒ _handleNotificationTap(userId, notification)</code>	<code>notifications_screen_v2.dart:304</code>
Handler Method	<code>Future<void> _handleNotificationTap(...)</code>	<code>notifications_screen_v2.dart:391</code>
Mark-as-Read	<code>await ref...markAsRead(notification.id)</code>	<code>notifications_screen_v2.dart:394-396</code>
Await Strategy	✅ AWAITS the Future	<code>notifications_screen_v2.dart:394</code>
Route Source	<code>notification.actionRoute</code> (NOT <code>data.route</code>)	<code>notifications_screen_v2.dart:402</code>

Aspect	Implementation	File:Line
Route Guard	<code>if (notification.actionRoute != null && mounted)</code>	<code>notifications_screen_v2.dart:401</code>
Navigation	<code>context.push(notification.actionRoute!)</code>	<code>notifications_screen_v2.dart:402</code>
Mark Error Catch	Controller catches, sets <code>state.error</code>	<code>notifications_controller_v2.dart:159-160</code>
Mark Error Display	✗ NOT displayed to user	N/A
Nav Error Catch	✗ NOT caught	N/A

ISSUES IDENTIFIED

Issue 1: Silent Failure on Mark-as-Read

```
// Controller catches error but doesn't rethrow
catch (e) {
  state = state.copyWith(error: e.toString()); // ← Stored but not shown
}

// UI awaits but doesn't check result
await ref...markAsRead(notification.id); // ← Completes "successfully" even on error
```

Impact: User taps notification, marking fails due to network/RLS, but still navigates. Notification stays unread with no user feedback.

Fix:

```
// Option 1: Rethrow in controller
catch (e) {
  state = state.copyWith(error: e.toString());
  rethrow; // ← Let UI handle it
}

// Option 2: Check state.error in UI
await ref...markAsRead(notification.id);
if (ref.read(notificationsControllerProvider(userId)).error != null) {
  ScaffoldMessenger.of(context).showSnackBar(
```

```
    SnackBar(content: Text('Failed to mark as read')),
  );
  return; // Don't navigate
}
```

Issue 2: No Optimistic Update

```
await ref...markAsRead(notification.id);
// Update will come via realtime subscription ← Takes 100-500ms
```

Impact: UI feels sluggish. Unread indicator stays visible during network round-trip.

Fix: Update local state immediately:

```
// In controller
final index = state.notifications.indexWhere((n) ⇒ n.id == notificationId);
if (index != -1) {
  final updated = [...state.notifications];
  updated[index] = updated[index].copyWith(isRead: true);
  state = state.copyWith(notifications: updated);
}
await _repository.markRead(...); // Then sync to backend
```

Issue 3: Navigation Not Guarded

```
context.push(notification.actionRoute!); // ← Can fail
```

Impact: Invalid routes cause crashes or error pages.

Fix:

```
try {
  context.push(notification.actionRoute!);
} on Exception catch (e) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Cannot open notification')),
  );
}
```

```
);  
}
```

✓ Good Practices Found

1. **Mounted check** prevents navigation after disposal
2. **Null check** prevents navigation without route
3. **Await strategy** ensures mark-as-read completes before navigation (though errors ignored)
4. **Dedicated field** `actionRoute` separate from generic `data` map

Step 5 — “Write + read” plumbing check (no schema changes)

Analysis of notifications_repository.dart

1) `markRead()` Implementation (lines 422-436)

```
Future<void> markRead({required String notificationId}) async {  
  try {  
    await _client  
      .from('notifications')  
      .update({  
        'is_read': true,  
        'read_at': DateTime.now().toIso8601String(),  
        'updated_at': DateTime.now().toIso8601String(),  
      })  
      .eq('id', notificationId);  
  } catch (e, stack) {  
    throw RepoException(  

```

```

    'Failed to mark notification as read',
    error: e,
    stackTrace: stack,
  );
}
}

```

- **Table:** `notifications`
- **Filter:** `.eq('id', notificationId)`
- **Columns Updated:** `is_read: true`, `read_at`, `updated_at`
- **Return Type:** `Future<void>` (does NOT return updated row)
- **Error Handling:** Catches and **rethrows as RepoException**

2) `list()` Implementation (lines 307-353)

```

// Start query
var query = _client
  .from('notifications')
  .select()
  .eq('user_id', userId);

// Apply type filter
if (typeFilter != null) {
  query = query.eq('type', typeFilter);
}

// Apply priority filter
if (priorityFilter != null) {
  query = query.eq('priority', priorityFilter);
}

// Apply is_read filter
if (isRead != null) {
  query = query.eq('is_read', isRead);
}

```

```
// Apply keyset pagination cursor
if (cursor != null) {
    final cursorTime = cursor.createdAt.toIso8601String();
    final cursorId = cursor.id;

    query = query.or(
        'created_at.lt.$cursorTime,and(created_at.eq.$cursorTime,id.lt.$cursorId)'
    );
}

// Order and limit
final response = await query
    .order('created_at', ascending: false)
    .order('id', ascending: false)
    .limit(limit);

return (response as List)
    .map((json) ⇒ NotificationItem.fromMap(json as Map<String, dynamic>))
    .toList();
```

- **Filters:** `.eq('user_id', userId)` + optional type/priority/isRead
- **Ordering:** `created_at DESC, id DESC`
- **Limit:** Default 20, configurable
- **Keyset Predicate:** `.or('created_at.lt.$cursorTime,and(created_at.eq.$cursorTime,id.lt.$cursorId)')`
- **Error Handling:** Catches and **rethrows as RepoException**

3) `subscribeUserNotifications()` Implementation (lines 518-591)

```
Stream<NotificationItem> subscribeUserNotifications(String userId) {
    final controller = StreamController<NotificationItem>.broadcast();

    // Clean up any existing subscription for this user
    final channelKey = 'notifications_{$userId}';
    _subscriptions[channelKey]?.unsubscribe();
```

```

// Create new realtime channel
final channel = _client
    .channel(channelKey)
    .onPostgresChanges(
        event: PostgresChangeEvent.insert,
        schema: 'public',
        table: 'notifications',
        filter: PostgresChangeFilter(
            type: PostgresChangeFilterType.eq,
            column: 'user_id',
            value: userId,
        ),
        callback: (payload) {
            try {
                final notification = NotificationItem.fromMap(
                    payload.newRecord,
                );
                controller.add(notification);
            } catch (e) {
                controller.addError(RepoException(
                    'Failed to parse INSERT payload',
                    error: e,
                ));
            }
        },
    )
    .onPostgresChanges(
        event: PostgresChangeEvent.update,
        schema: 'public',
        table: 'notifications',
        filter: PostgresChangeFilter(
            type: PostgresChangeFilterType.eq,
            column: 'user_id',
            value: userId,
        ),
        callback: (payload) {
            try {
                final notification = NotificationItem.fromMap(

```

```

        payload.newRecord,
    );
    controller.add(notification);
} catch (e) {
    controller.addError(RepoException(
        'Failed to parse UPDATE payload',
        error: e,
    ));
}
},
)
.subscribe();

_subscriptions[channelKey] = channel;

return controller.stream;
}

```

- **Channel Key:** `'notifications_$userId'`
- **Schema/Table:** `public.notifications`
- **Filter:** `user_id = userId`
- **Events:** INSERT and UPDATE
- **Callback:** Parses `payload.newRecord` → `NotificationItem.fromMap()` → `controller.add()`
- **State Update:** Controller emits to stream consumed by `NotificationsController._setupRealtimeSubscription()`

4) Critical Issues Found

Pessimistic Update Strategy

- **Repository** throws exceptions properly via `RepoException`
- **Controller** catches exceptions but **swallows them** (sets `state.error` without rethrowing)
- **UI** calls `await controller.markAsRead()` which completes "successfully" even on failure

- **Result:** UI waits for server response (pessimistic), then navigation happens regardless of success/failure

Flow:

1. User taps notification → `_handleNotificationTap()` awaits `markAsRead()`
2. Controller awaits repository `markRead()` (network round-trip: 100-500ms)
3. Repository throws `RepoException` on failure
4. Controller **catches and swallows** exception (line 156-159 in controller)
5. UI's `await` completes without error
6. Navigation proceeds even if database update failed
7. Unread indicator stays visible until realtime UPDATE event arrives (another 100-500ms)

Missing Error Propagation in Controller

From `notifications_controller_v2.dart` line 155-161:

```
Future<void> markAsRead(String notificationId) async {
  try {
    await _repository.markRead(notificationId: notificationId);
    // State will be updated via realtime subscription
  } catch (e, stack) {
    state = state.copyWith(error: e.toString());
    // ❌ Does NOT rethrow - UI thinks operation succeeded
  }
}
```

Fix needed: Rethrow after setting `state.error` so UI can handle failure properly.